Try

Try · Parse config int with recovery · Build with Waqar

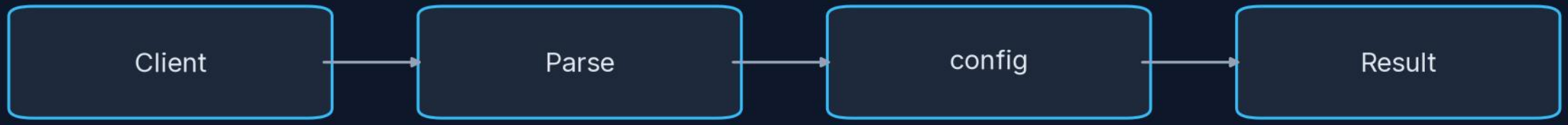
Today's Hook

- Today's force: Parse config int with recovery
- We earn Try by feeling the naive pain first.
- Demo main prints a real JDK 25 trace (happy + failure).
- Key types: Try, Success, Failure, Tries

The Problem

- Scenario: Parse config int with recovery
- Constraints: solve «Parse config int with recovery» with Try — keep the demo runnable and the failure path observable.
- Naive pain: Naive version fails to cleanly solve: Parse config int with recovery
- Success looks like: caught

Structure



Try: Parse config int with recovery

Real-World Scenario

- Running example: Parse config int with recovery
- Key types: Try, Success, Failure, Tries
- Happy-path stdout cue: caught
- Failure-path stdout cue: Failure path is asserted in ProblemTest (injected fault / invalid input)

Naive Code

```
package com.buildwithwaqar.patterns.functional.try_.problem;

import java.util.*;

public final class NaiveCfg {
  public static int port(String s){ return Integer.parseInt(s); }
}
```

Why Naive Hurts

- Naive version fails to cleanly solve: Parse config int with recovery
- Without Try, call sites absorb the wrong details.
- Failure mode to show on camera: Failure path is asserted in ProblemTest (injected fault / invalid input)
- New requirements force shotgun edits across the naive code.

Solution

```java
package com.buildwithwaqar.patterns.functional.try_.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.concurrent.locks.*;
import java.util.function.*;

sealed interface Try<A> permits Success, Failure { A getOrElse(A fallback); boolean isSuccess(); }
record Success<A>(A value) implements Try<A> { public A getOrElse(A f){return value;} public boolean isSuccess(){return true;} }
record Failure<A>(Throwable error) implements Try<A> { public A getOrElse(A f){return f;} public boolean isSuccess(){return false;} }
final class Tries {
  static <A> Try<A> of(Callable<A> c){ try { return new Success<>(c.call()); } catch(Exception e){ return new Failure<>(e); } }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.functional.try_.problem;
import java.util.*; import java.util.concurrent.*; import java.util.concurrent.atomic.*;
public final class ProblemDemo {
  public static void main(String[] args) throws Exception {
    System.out.println("=== Try: parse config int with recovery ===");
    System.out.println("--- naive ---");
try { System.out.println(NaiveCfg.port("x")); } catch(Exception e){ System.out.println("caught"); }
System.out.println("--- try ---");
System.out.println(Tries.of(() -> Integer.parseInt("8080")).getOrElse(-1));
System.out.println(Tries.of(() -> Integer.parseInt("x")).getOrElse(8080));
  }
}
```

Walkthrough Output

```
=== Try: parse config int with recovery ===
--- naive ---
caught
--- try ---
8080
8080
```

Step-by-Step Trace

- Match stdout to stages — look for: caught
- Then the injected failure: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Re-run: java -cp ... 44-try ProblemDemo (see episode folder).
- Pause the video and predict the next printed line.

Modern Java 25 / Zazr

```java
package com.buildwithwaqar.patterns.functional.try_.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.concurrent.locks.*;
import java.util.function.*;

sealed interface Try<A> permits Success, Failure { A getOrElse(A fallback); boolean isSuccess(); }
record Success<A>(A value) implements Try<A> { public A getOrElse(A f){return value;} public boolean isSuccess(){return true;} }
record Failure<A>(Throwable error) implements Try<A> { public A getOrElse(A f){return f;} public boolean isSuccess(){return false;} }
final class Tries {
  static <A> Try<A> of(Callable<A> c){ try { return new Success<>(c.call()); } catch(Exception e){ return new Failure<>(e); } }
}
```

Trade-offs

- Use Try when «Parse config int with recovery» repeats and a simpler feature does not erase it.
- Refuse extra types when a sealed switch or a function value already suffices.
- Teaching demos omit production metrics, retries, and multi-region clocks.
- Interview soundbite: state the force behind Try in product language.

Related Patterns

- Name the pattern usually confused with Try and separate the forces.
- Sketch the roles for «Parse config int with recovery» from memory in sixty seconds.
- Point to the next episode in this season on the playlist map.
- Link this episode's series/ folder in the YouTube description.

Recap

- Problem: Parse config int with recovery
- Naive pain → Try boundary → Demo stdout.
- Remember the failure cue: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Break a ProblemTest assertion, watch it fail, restore it.